Diagrammes UML — version 2

Plateforme de Gestion du Département Informatique (PGDI2026)

Module Génie Logiciel - Master M2I - Faculté des Sciences d'Oujda

v02 (28/10/2026) : modules 6 (offres externes via API) et 7 (veille externe) — diagrammes mis à jour marqués *

Diagrammes de cas d'utilisation

UC 0 - Vue globale * p. 2
UC 1 - Visiteur Authentification * p. 3
UC 2 - Administrateur * p. 4
UC 3 - Etudiant * p. 5
UC 4 - Enseignant p. 6
UC 5 - Responsable * p. 7
UC 6 - Stages PFE * p. 8

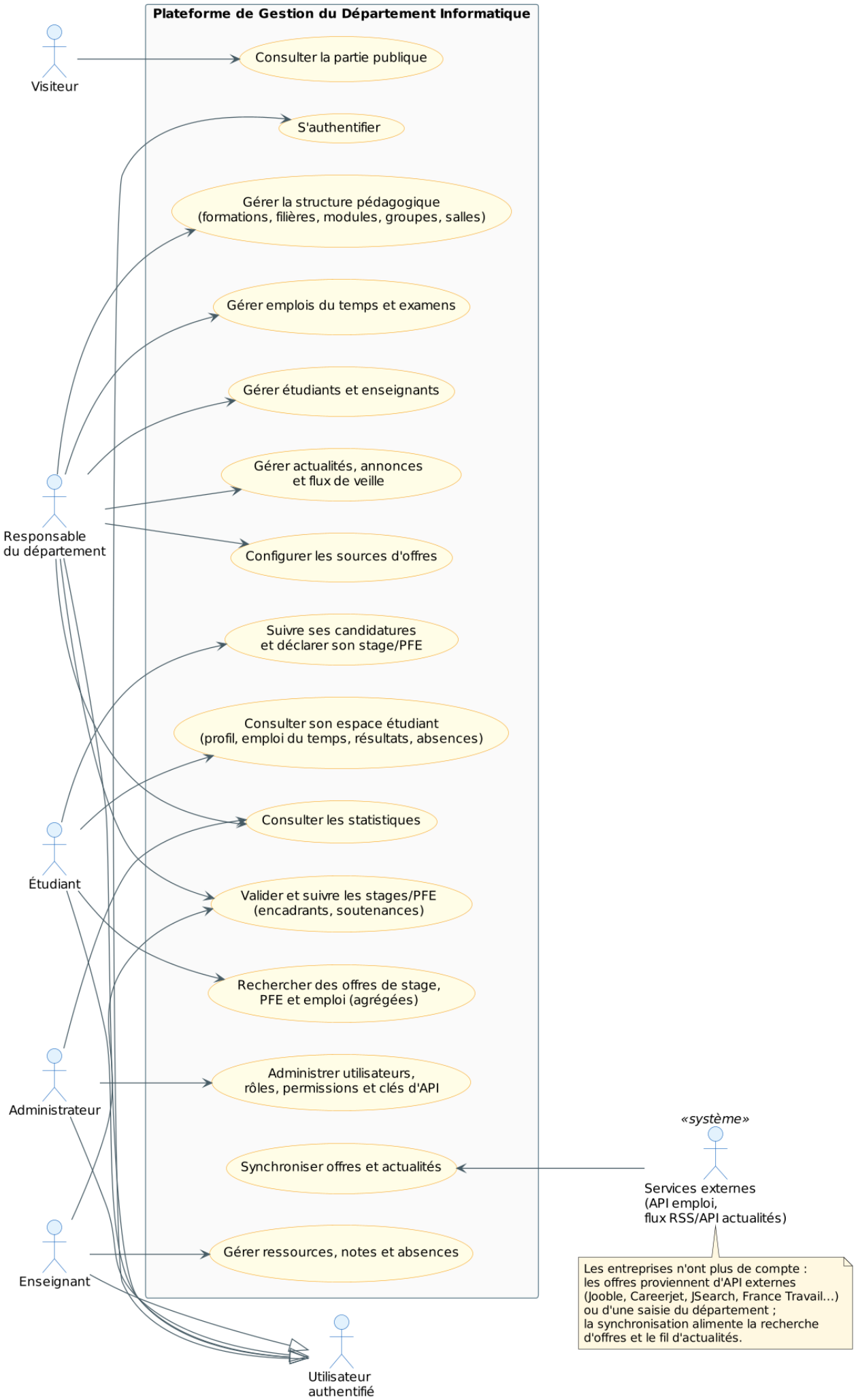
Diagrammes de classes

CL 0 - Vue globale * p. 9
CL 1 - Securite Public * p. 10
CL 2 - Structure pedagogique p. 11
CL 3 - Etudiants Enseignants p. 12
CL 4 - Enseignement Planning Examens p. 13
CL 5 - Notes Absences Ressources p. 14
CL 6 - Stages PFE * p. 15
CL 7 - Enumerations * p. 16

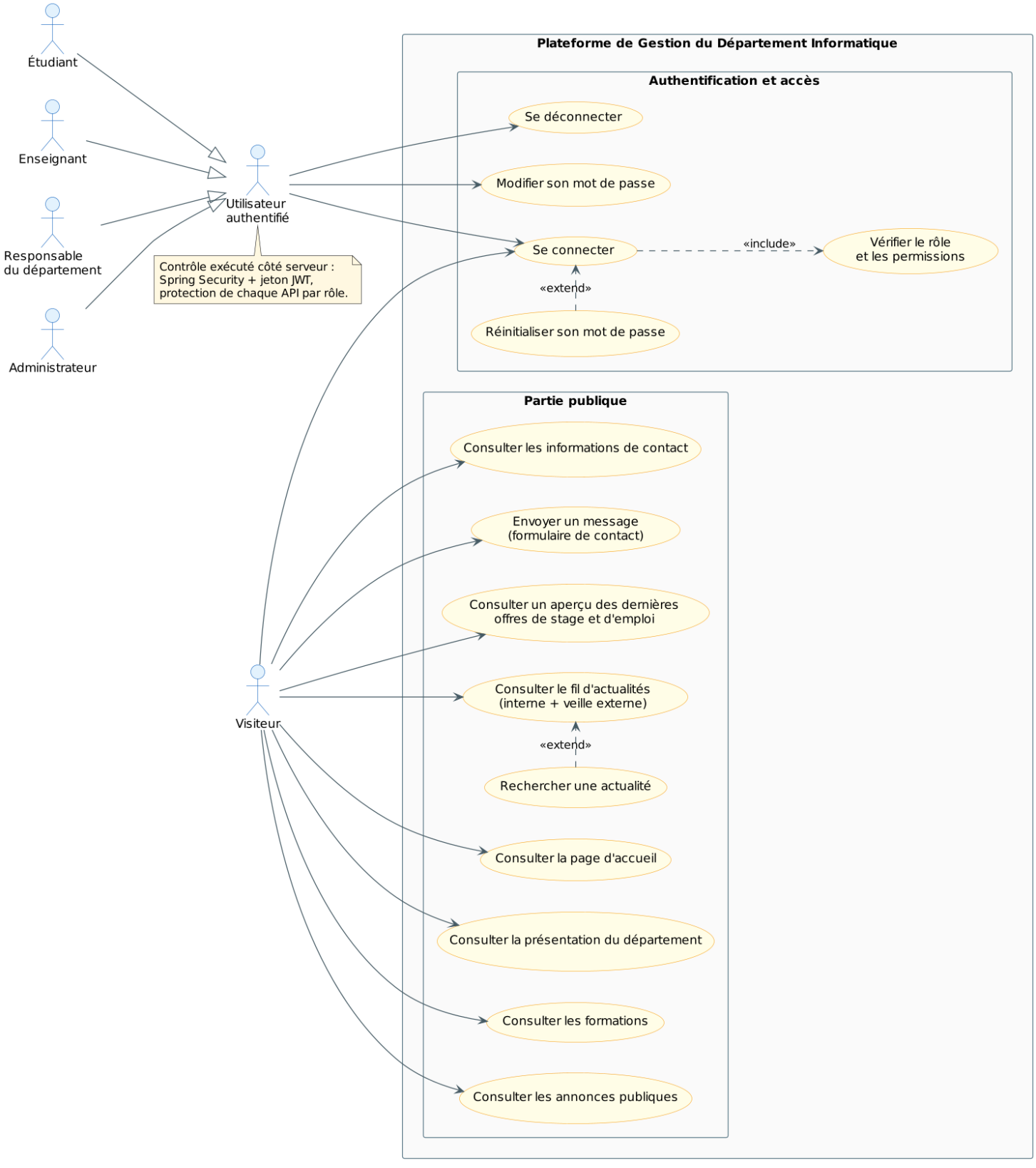
Diagrammes de séquence

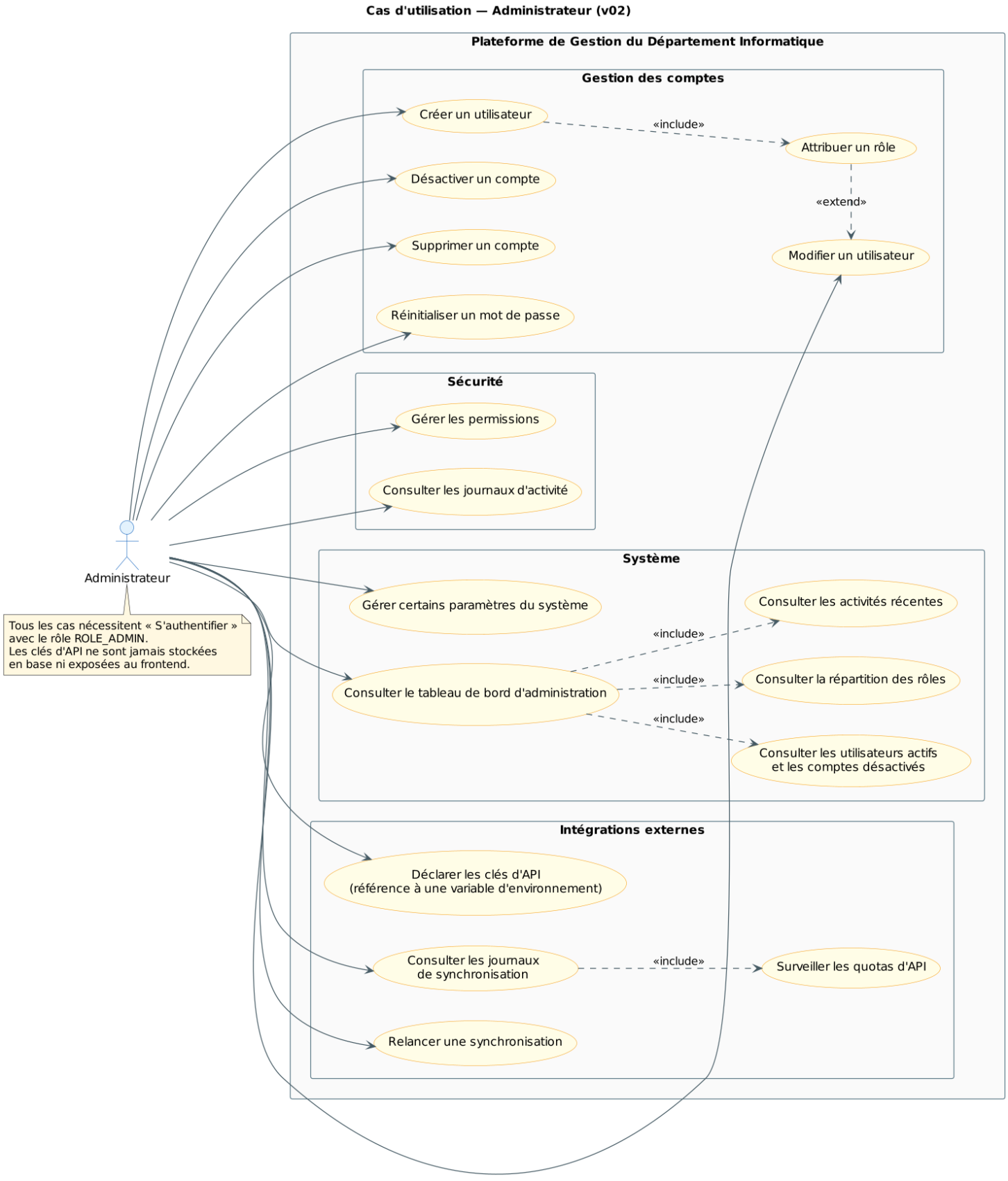
SQ 1 - Authentification JWT p. 17
SQ 2 - Reinitialisation mot de passe p. 18
SQ 3 - Creation utilisateur admin p. 19
SQ 4 - Creation seance emploi du temps p. 20
SQ 5 - Saisie et consultation des notes p. 21
SQ 6 - Gestion des absences p. 22
SQ 7 - Synchronisation des offres externes * p. 23
SQ 8 - Recherche favoris alertes et suivi * p. 24
SQ 9 - Declaration validation et encadrement du stage * p. 25
SQ 10 - Planification de la soutenance * p. 26
SQ 11 - Actualites internes et veille externe * p. 27

Diagramme de cas d'utilisation — Vue globale de la plateforme (v02)

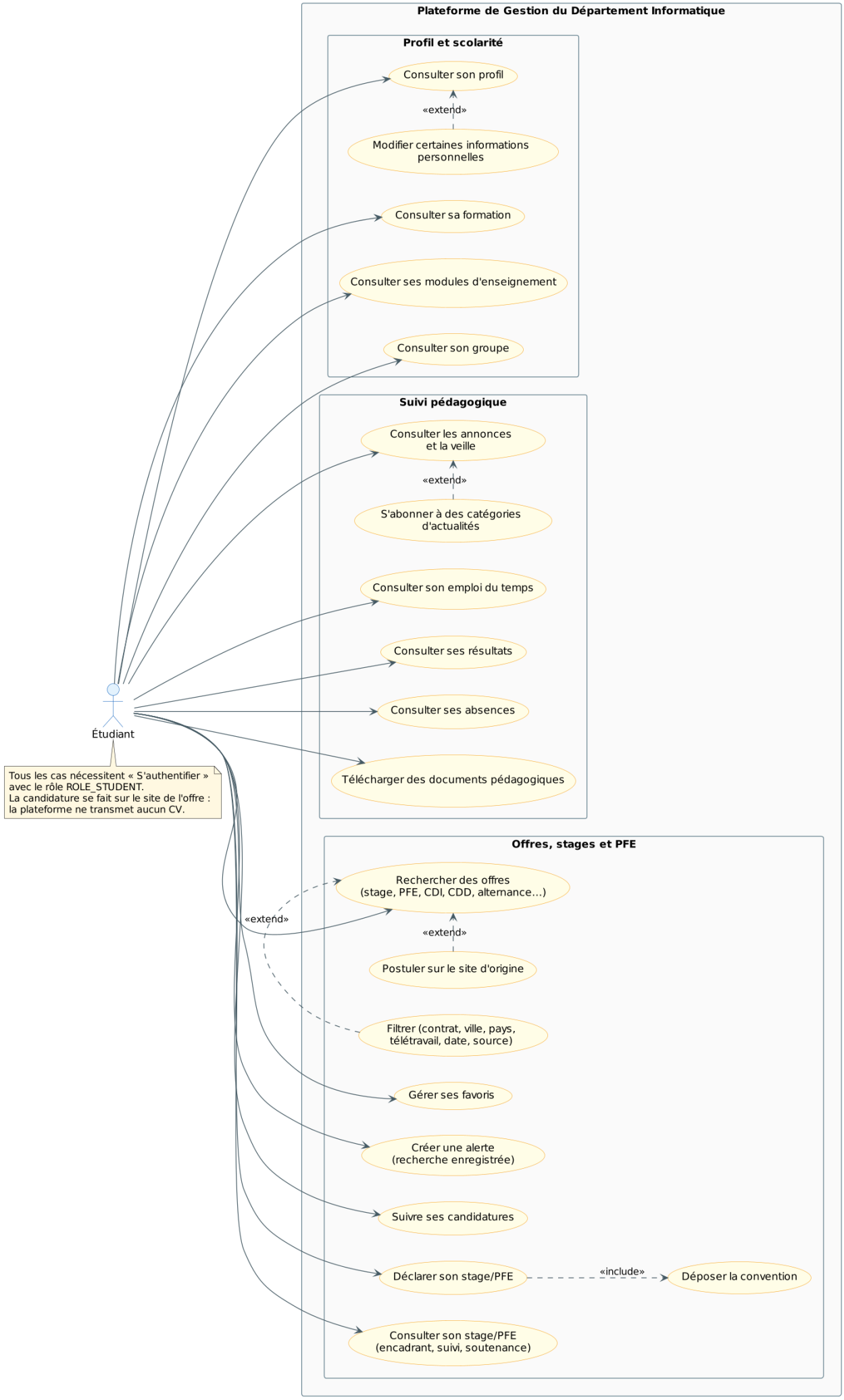


Cas d'utilisation — Visiteur et authentification (v02)

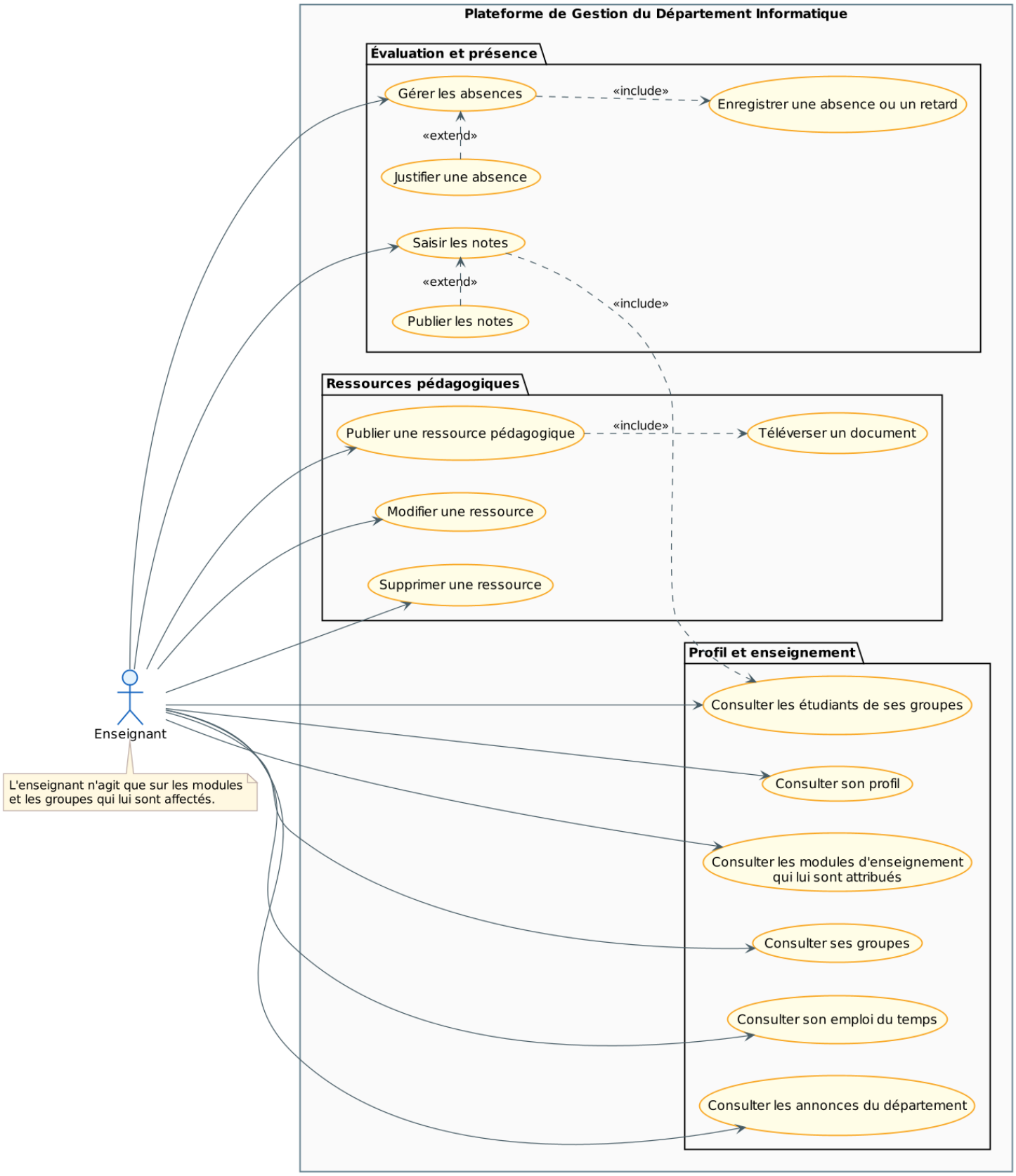



Cas d'utilisation — Administrateur (v02)
Plateforme de Gestion du Département Informatique
Gestion des comptes
Créer un utilisateur
«include»
Attribuer un rôle
Désactiver un compte
«extend»
Supprimer un compte
Modifier un utilisateur
Réinitialiser un mot de passe
Sécurité
Gérer les permissions
Consulter les journaux d'activité
Système
Gérer certains paramètres du système
Consulter les activités récentes
«include»
Consulter le tableau de bord d'administration
«include»
Consulter la répartition des rôles
«include»
Consulter les utilisateurs actifs et les comptes désactivés
Intégrations externes
Déclarer les clés d'API (référence à une variable d'environnement)
Consulter les journaux de synchronisation
«include»
Surveiller les quotas d'API
Relancer une synchronisation
Administrateur
Tous les cas nécessitent « S'authentifier » avec le rôle ROLE_ADMIN.
Les clés d'API ne sont jamais stockées en base ni exposées au frontend.

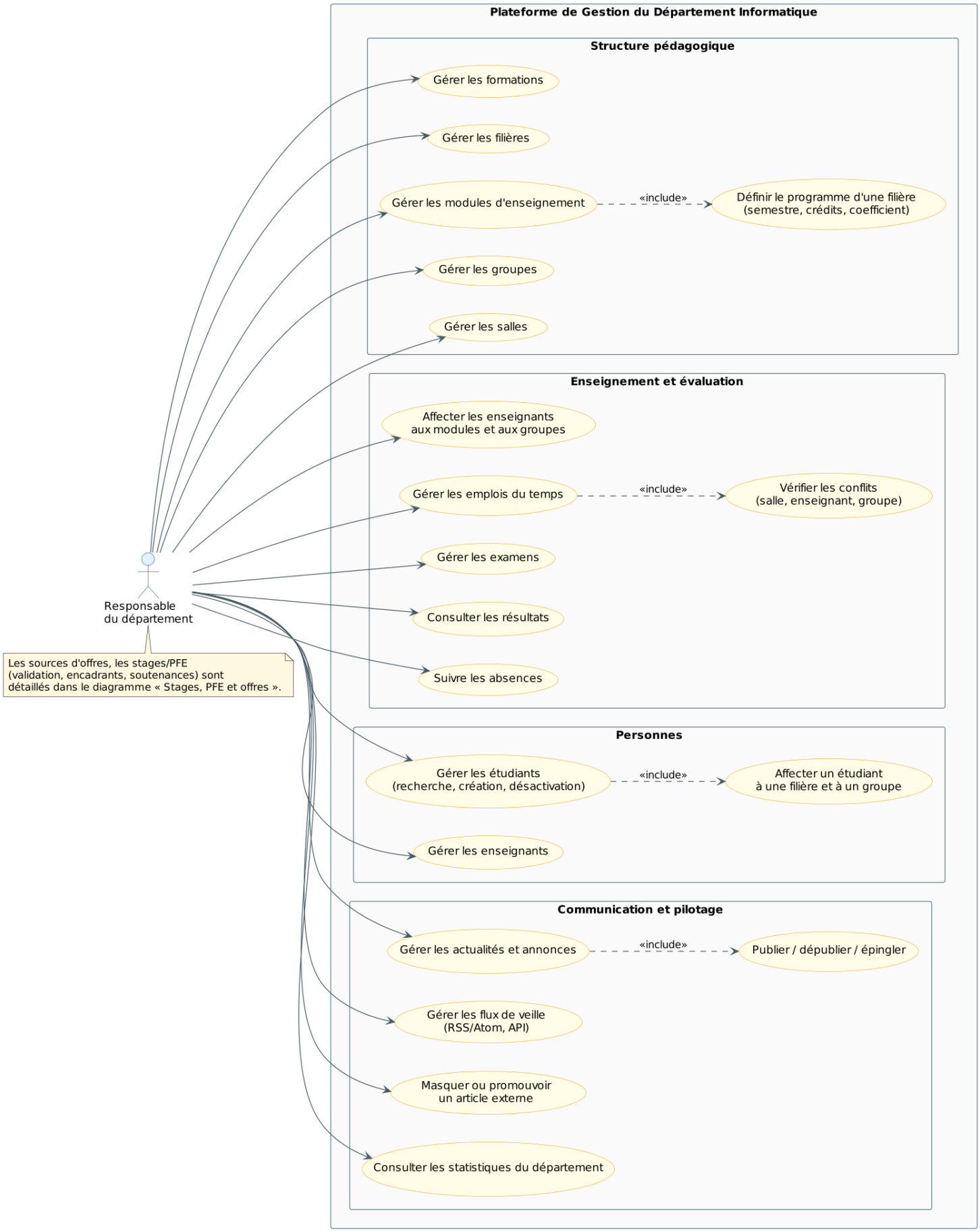
Cas d'utilisation — Espace étudiant (v02)



Cas d'utilisation — Espace enseignant



Cas d'utilisation — Responsable du département (gestion pédagogique et administrative, v02)



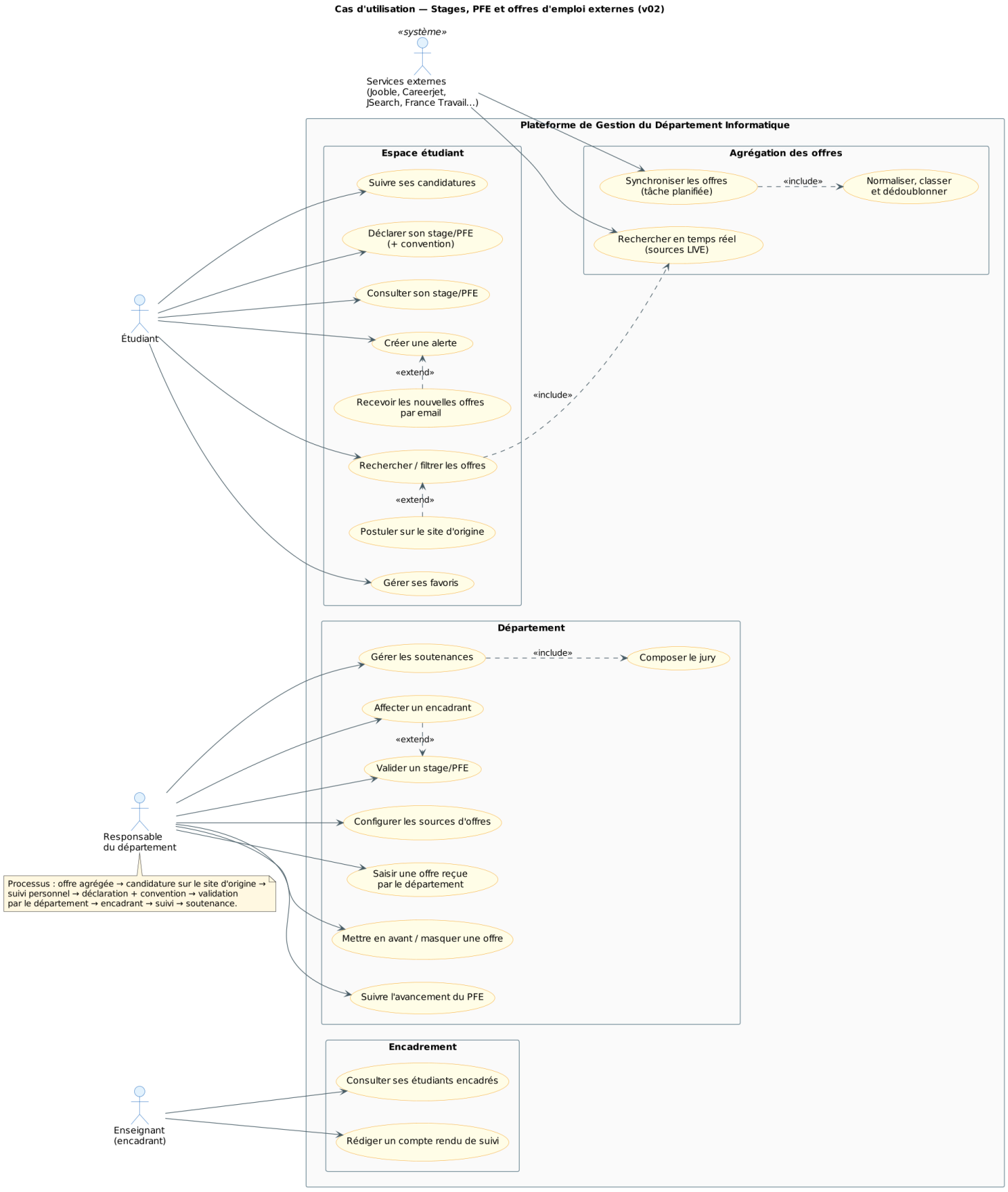

Cas d'utilisation — Stages, PFE et offres d'emploi externes (v02)
«système»
Services externes
(Jooble, Careerjet,
JSearch, France Travail…)
Plateforme de Gestion du Département Informatique
Espace étudiant
Suivre ses candidatures
Déclarer son stage/PFE
(+ convention)
Consulter son stage/PFE
Créer une alerte
«extend»
Recevoir les nouvelles offres
par email
Rechercher / filtrer les offres
«extend»
Postuler sur le site d'origine
Gérer ses favoris
Agrégation des offres
Synchroniser les offres
(tâche planifiée)
«include»
Normaliser, classer
et dédoublonner
Rechercher en temps réel
(sources LIVE)
«include»
Étudiant
Département
Gérer les soutenances
«include»
Composer le jury
Affecter un encadrant
«extend»
Valider un stage/PFE
Configurer les sources d'offres
Saisir une offre reçue
par le département
Mettre en avant / masquer une offre
Suivre l'avancement du PFE
Responsable
du département
Processus : offre agrégée → candidature sur le site d'origine →
suivi personnel → déclaration + convention → validation
par le département → encadrant → suivi → soutenance.
Encadrement
Consulter ses étudiants encadrés
Rédiger un compte rendu de suivi
Enseignant
(encadrant)

Diagramme de classes — Vue globale des entités (sans attributs, v02)

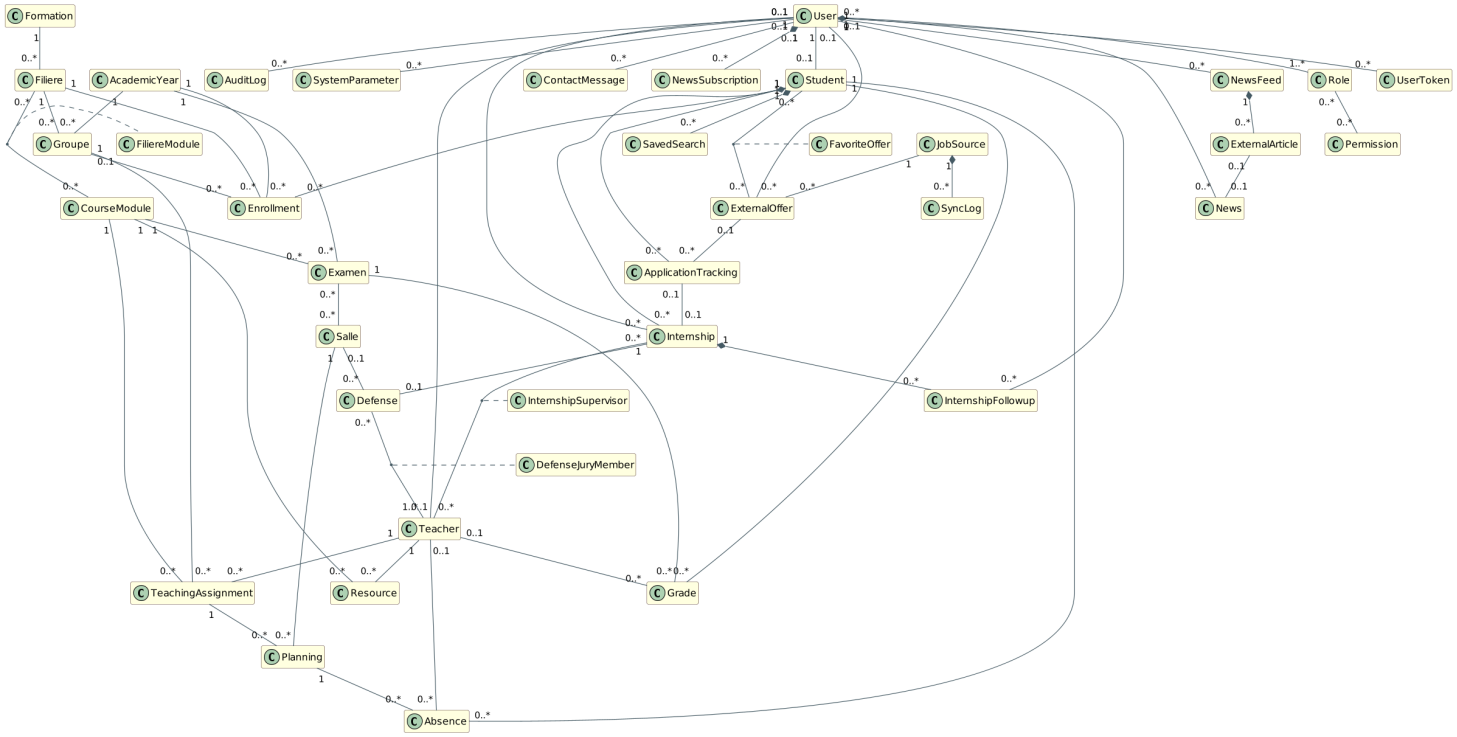


Diagramme de classes 1/6 — Sécurité, administration, partie publique et veille (v02)

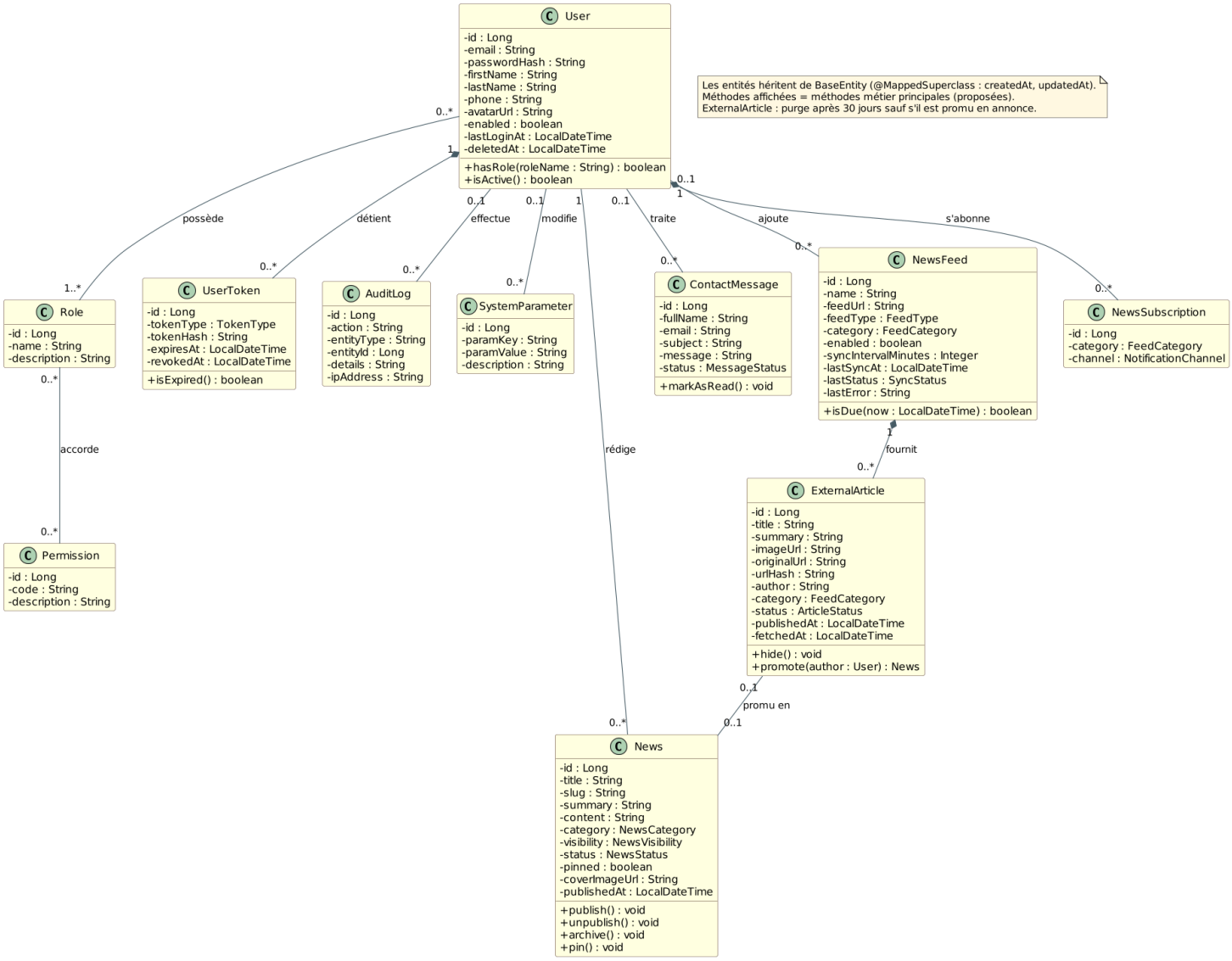
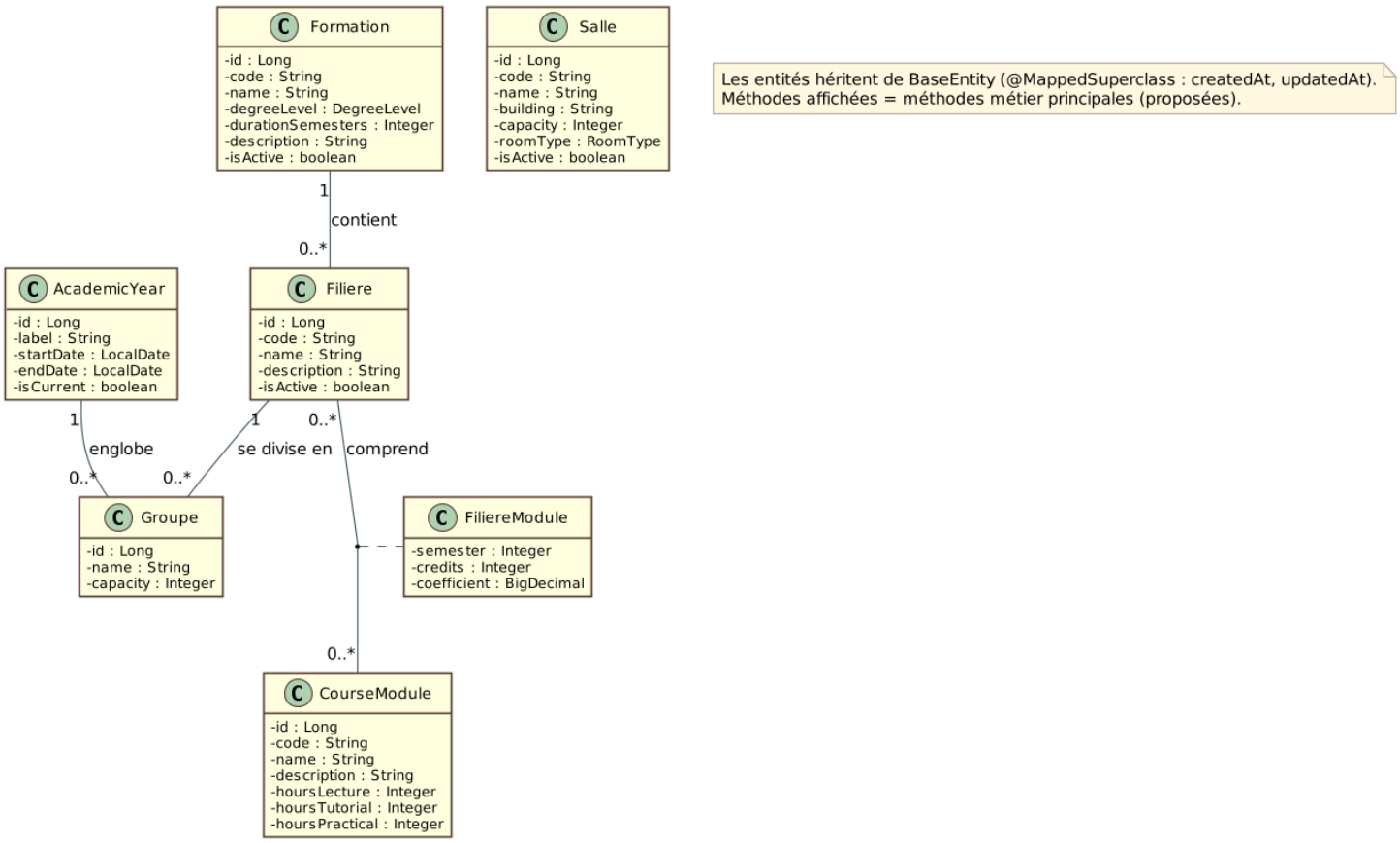


Diagramme de classes 2/6 — Structure pédagogique



Les entités héritent de BaseEntity (@MappedSuperclass : createdAt, updatedAt).
Méthodes affichées = méthodes métier principales (proposées).

Diagramme de classes 3/6 — Étudiants, enseignants, inscriptions

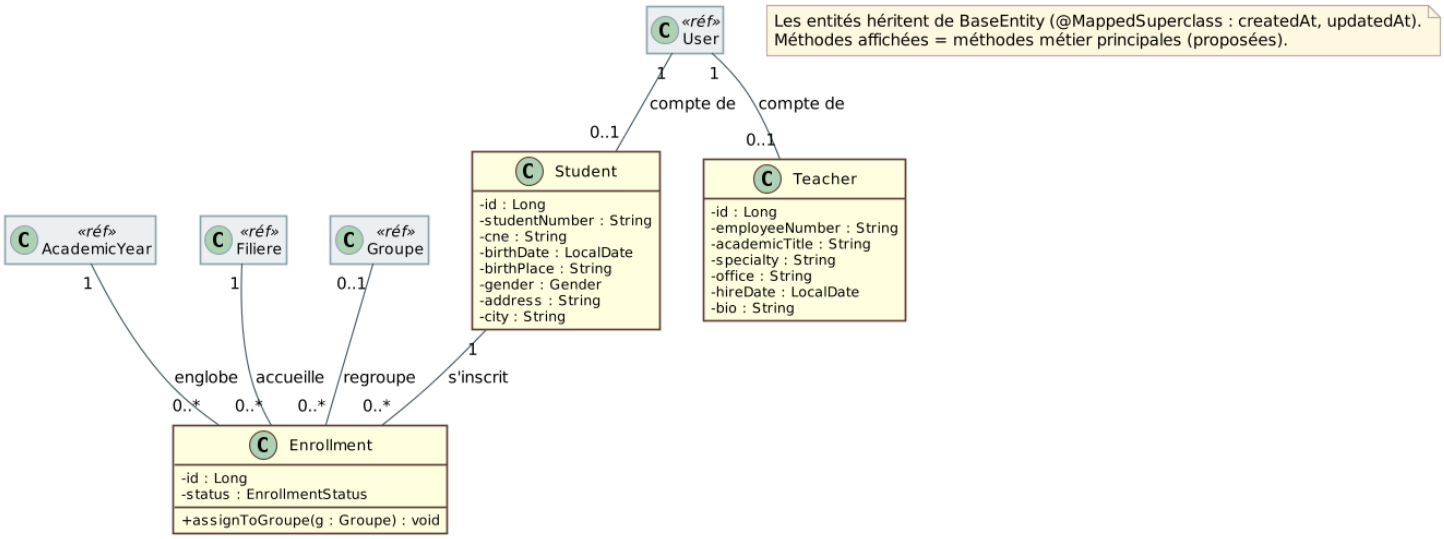
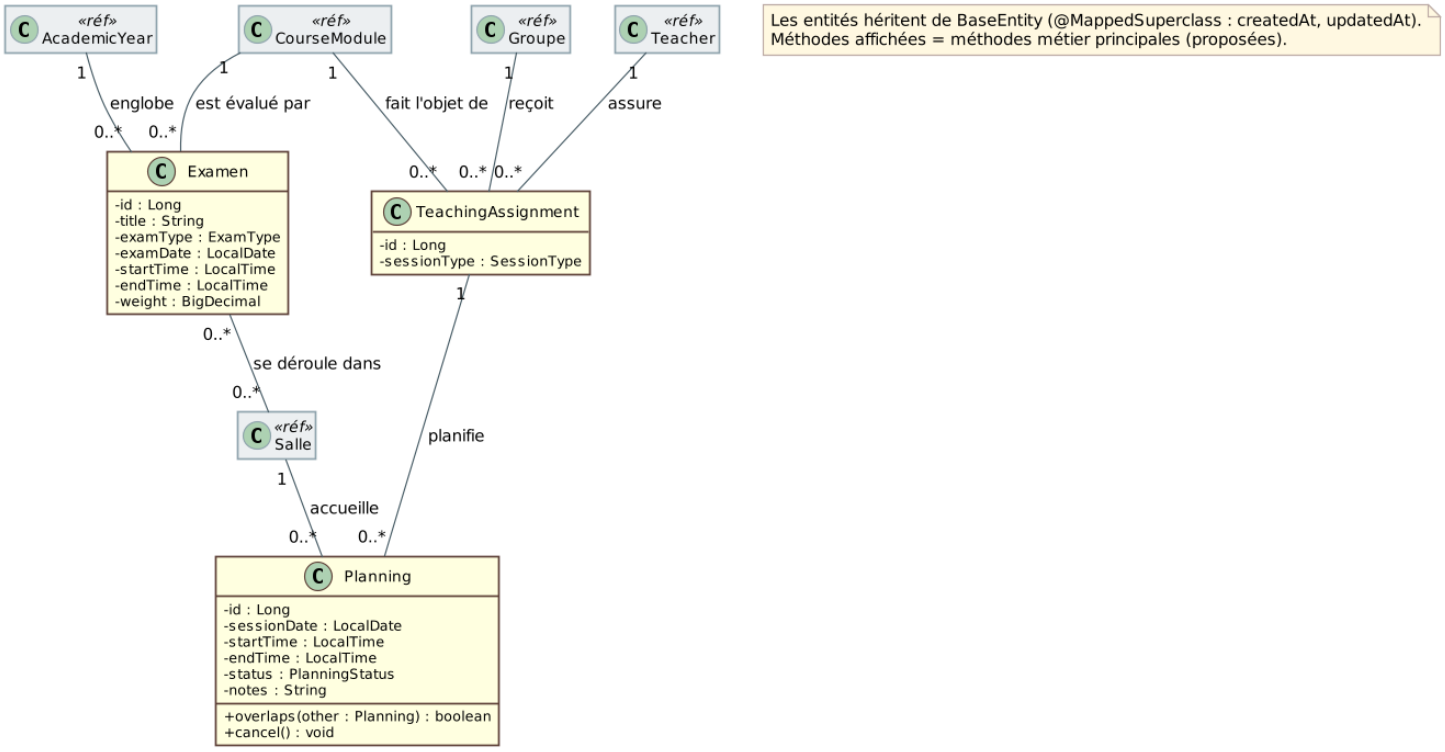


Diagramme de classes 4/6 — Enseignement, emploi du temps, examens



Les entités héritent de BaseEntity (@MappedSuperclass : createdAt, updatedAt).
Méthodes affichées = méthodes métier principales (proposées).

Diagramme de classes 5/6 — Notes, absences, ressources

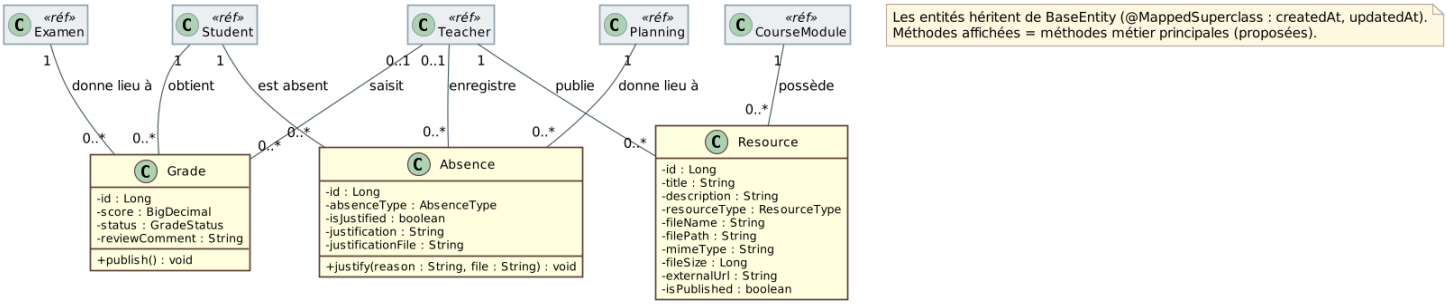


Diagramme de classes 6/6 — Stages, PFE et offres externes (v02)

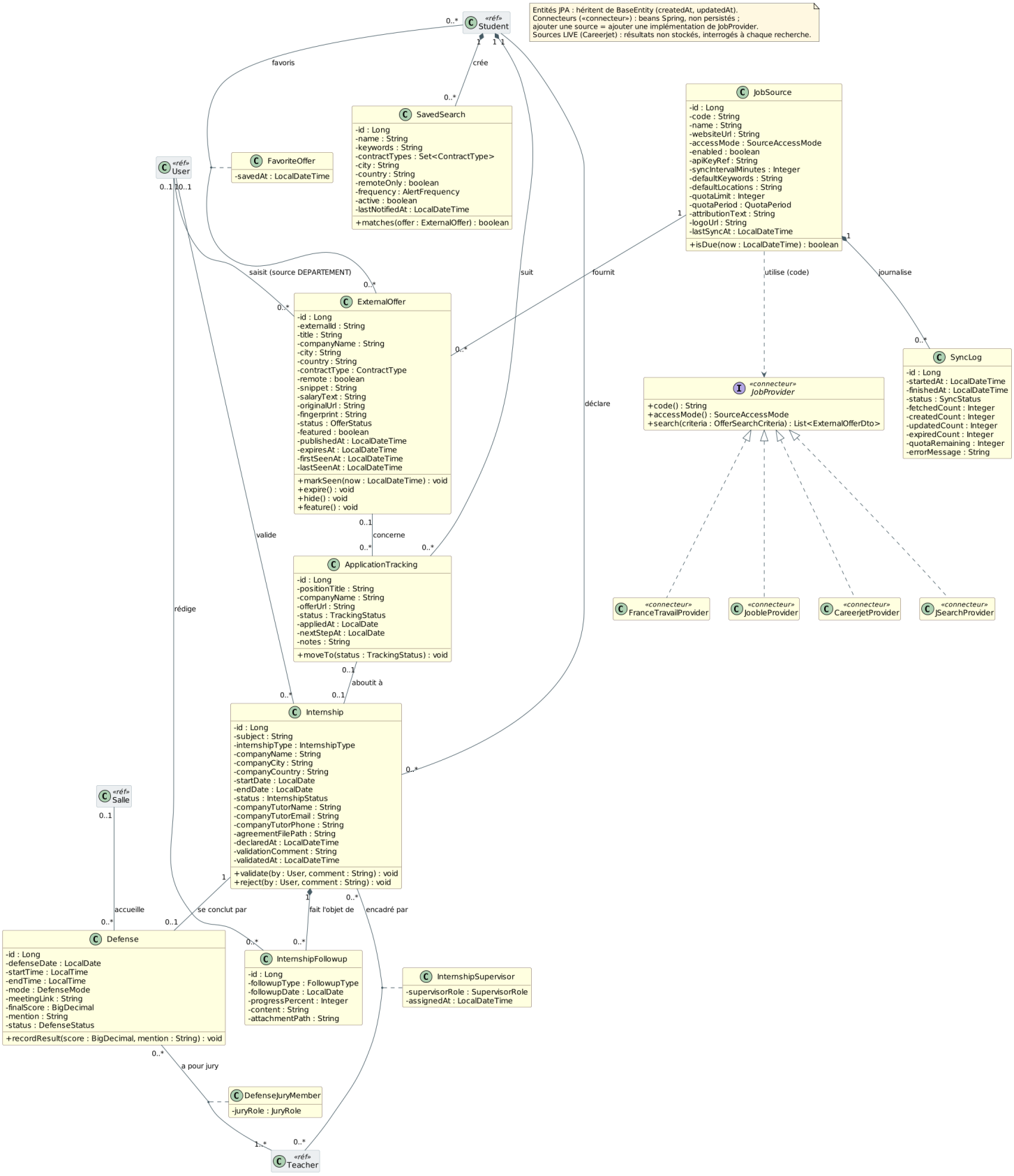
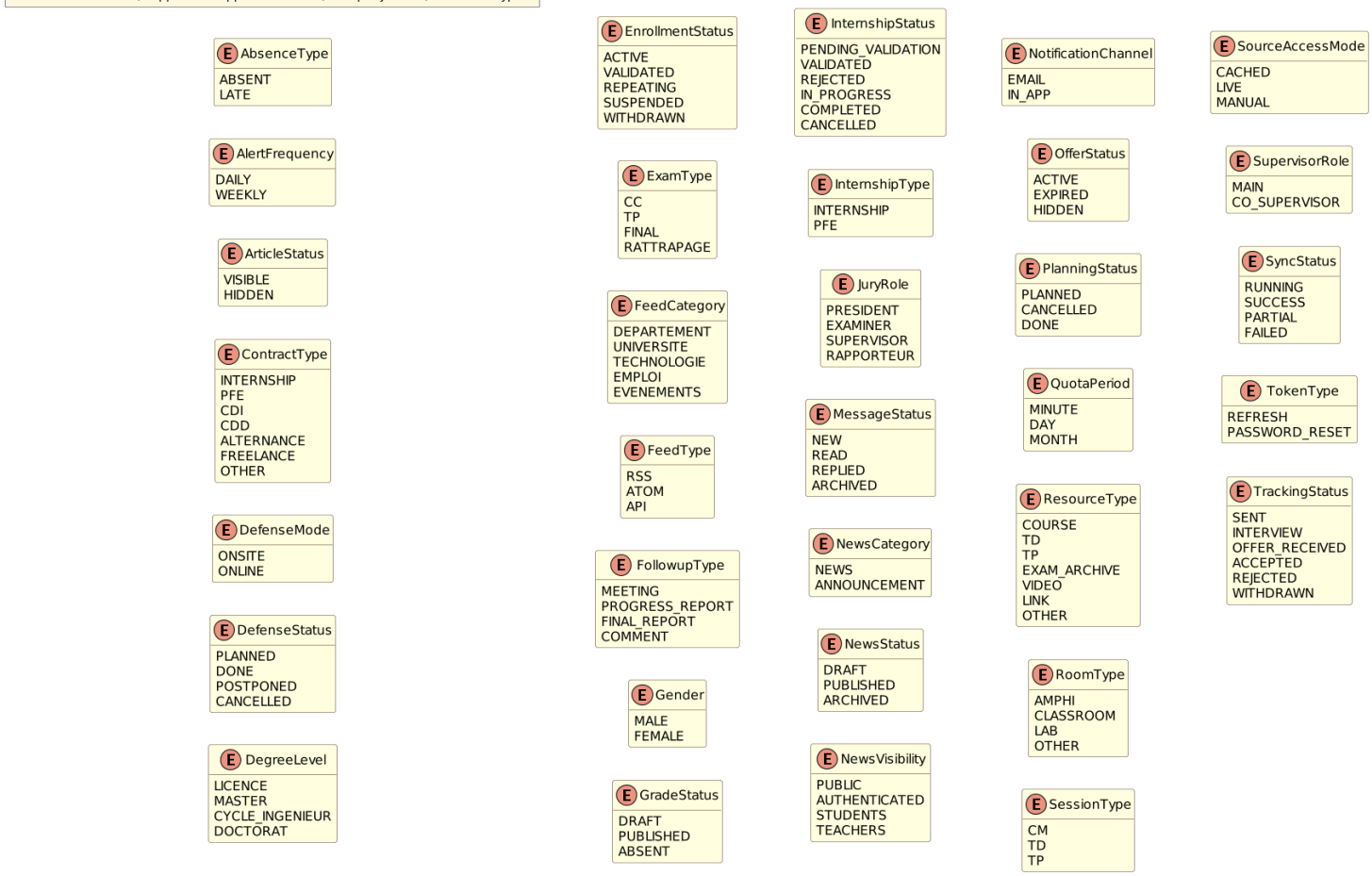
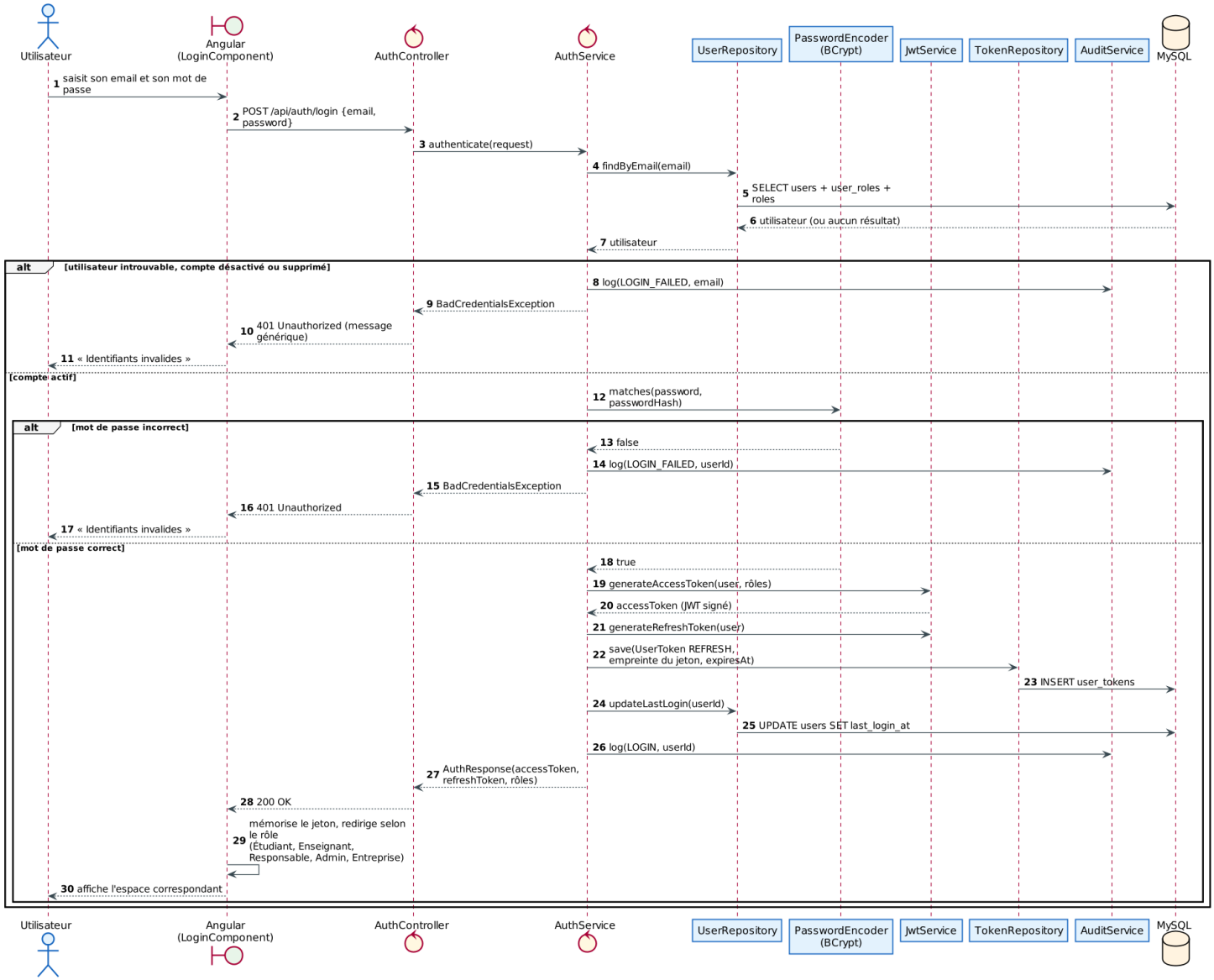


Diagramme de classes — Énumérations (@Enumerated(EnumType.STRING), v02)

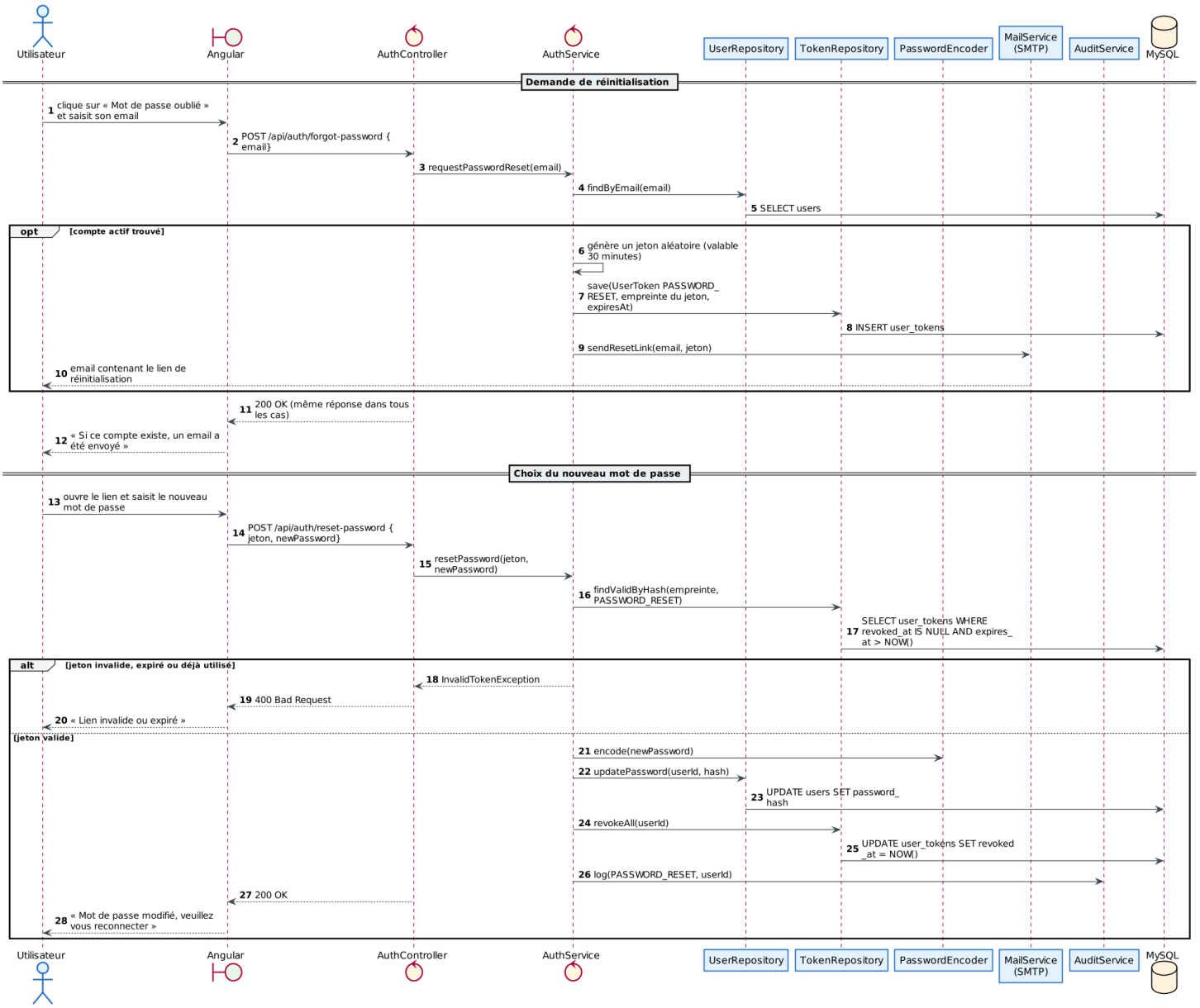
v02 : ajoutées AlertFrequency, ArticleStatus, ContractType, FeedCategory, FeedType, NotificationChannel, QuotaPeriod, SourceAccessMode, SyncStatus, TrackingStatus ; OfferStatus redéfinie ; supprimées ApplicationStatus, CompanyStatus, DocumentType.



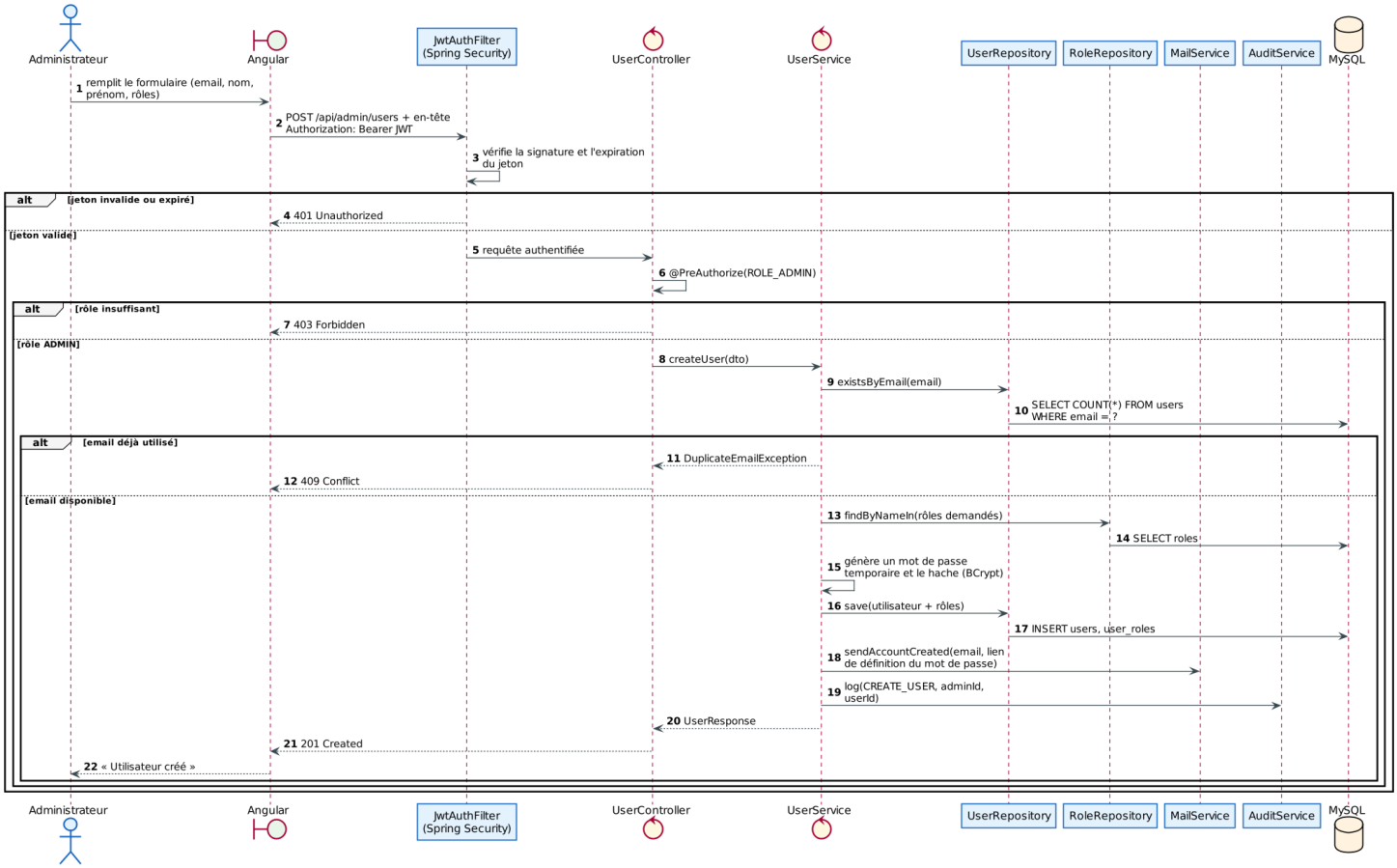
Séquence — Authentification (connexion avec jeton JWT)



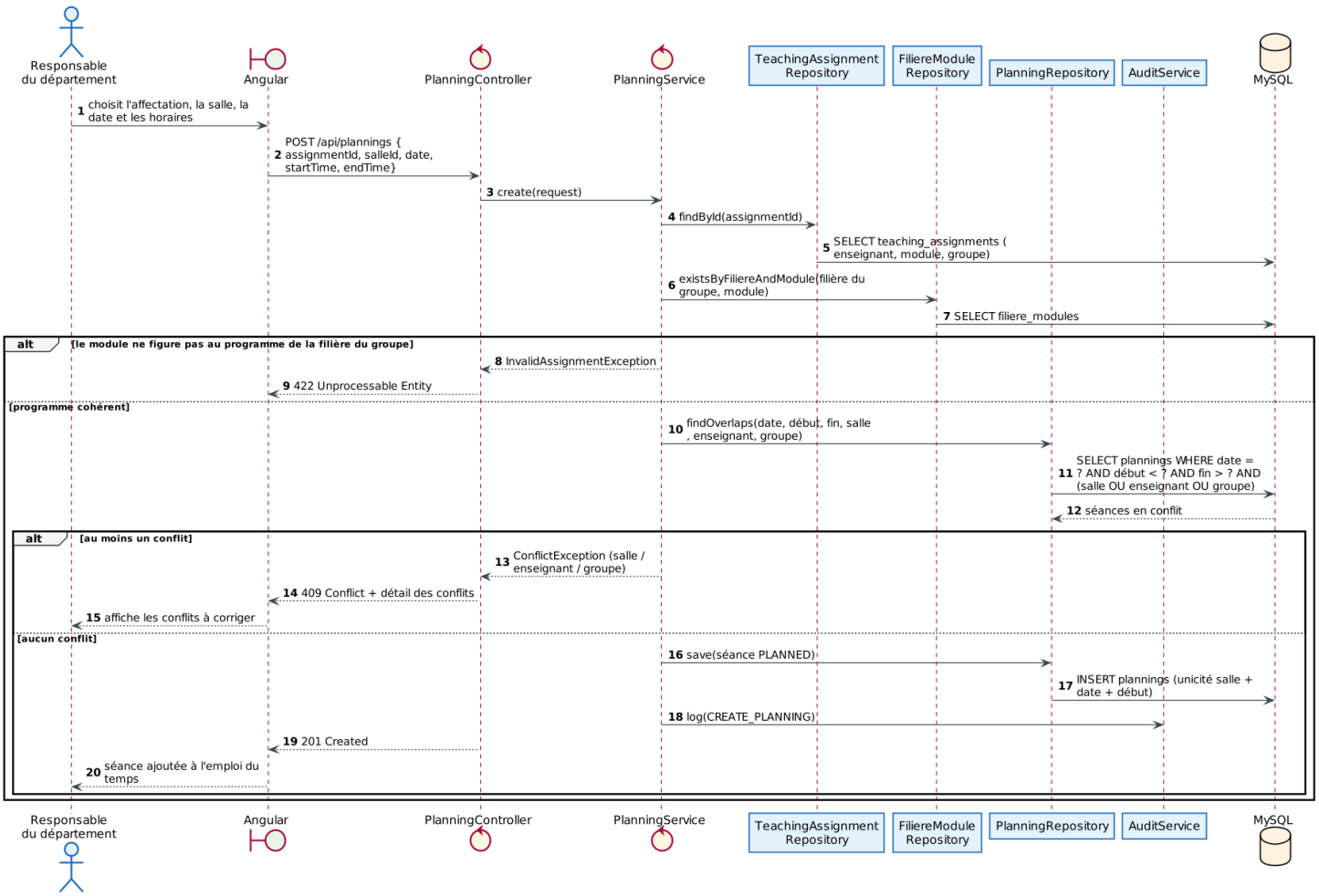
Séquence — Réinitialisation du mot de passe

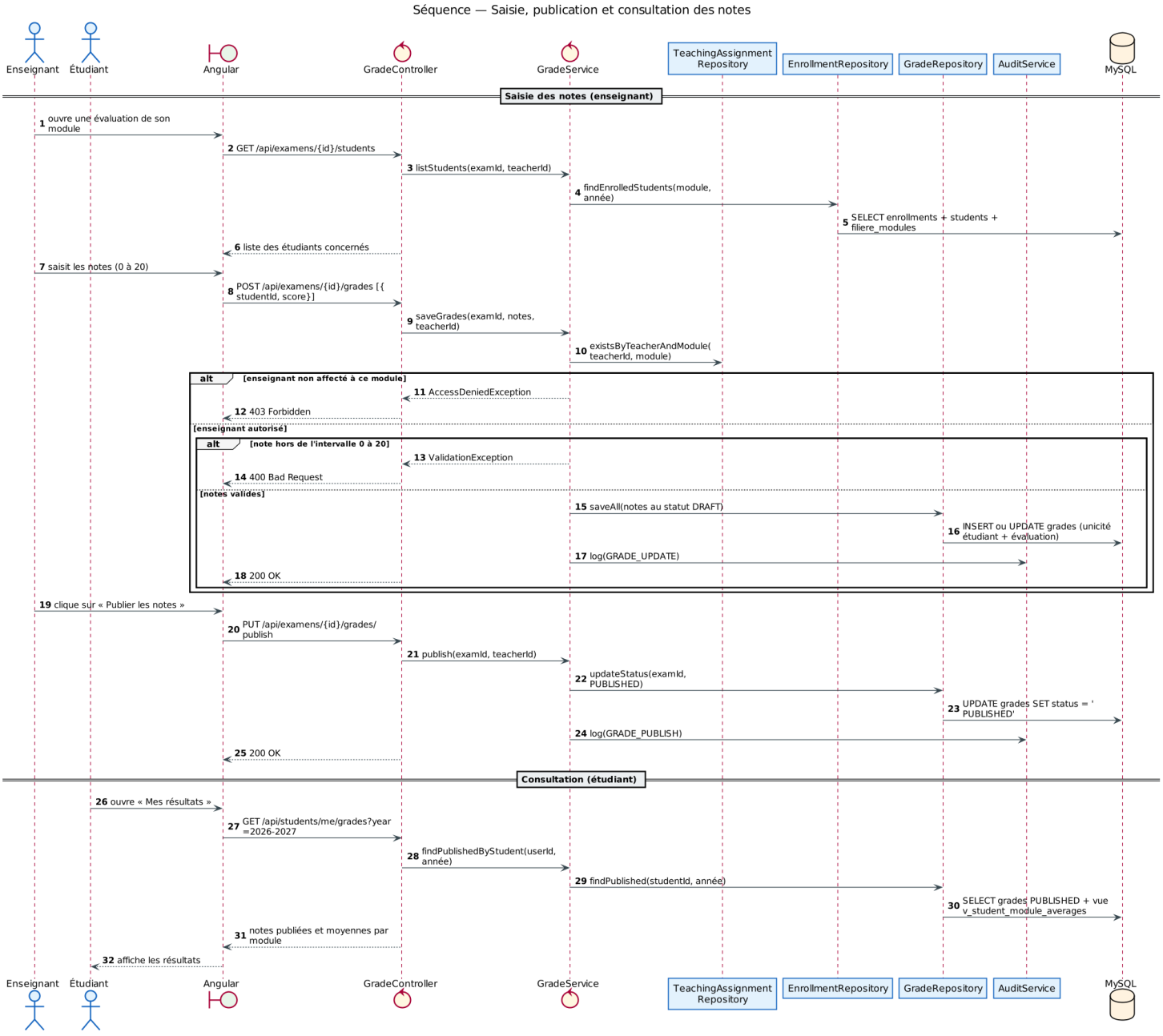


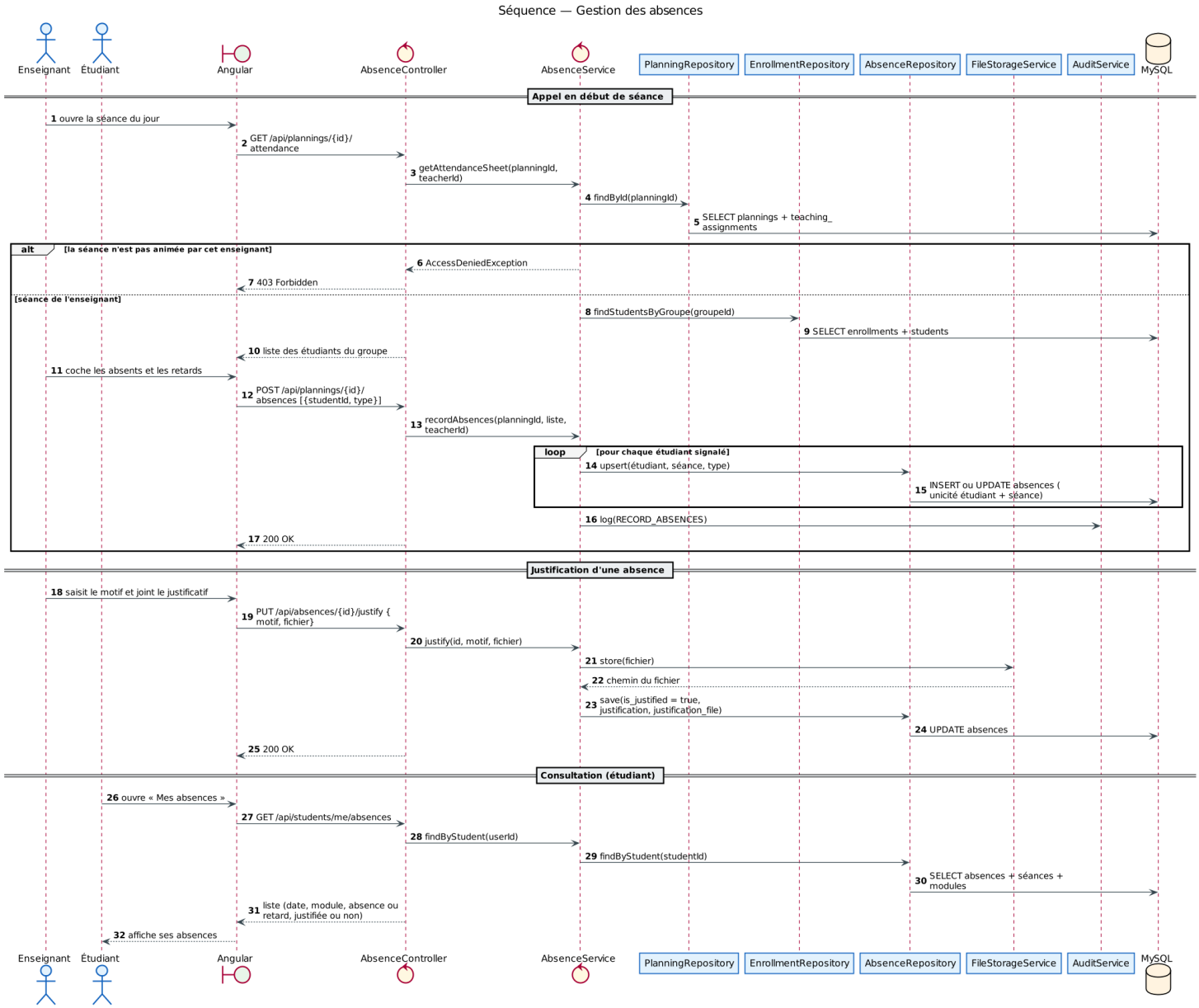
Séquence — Création d'un utilisateur par l'administrateur



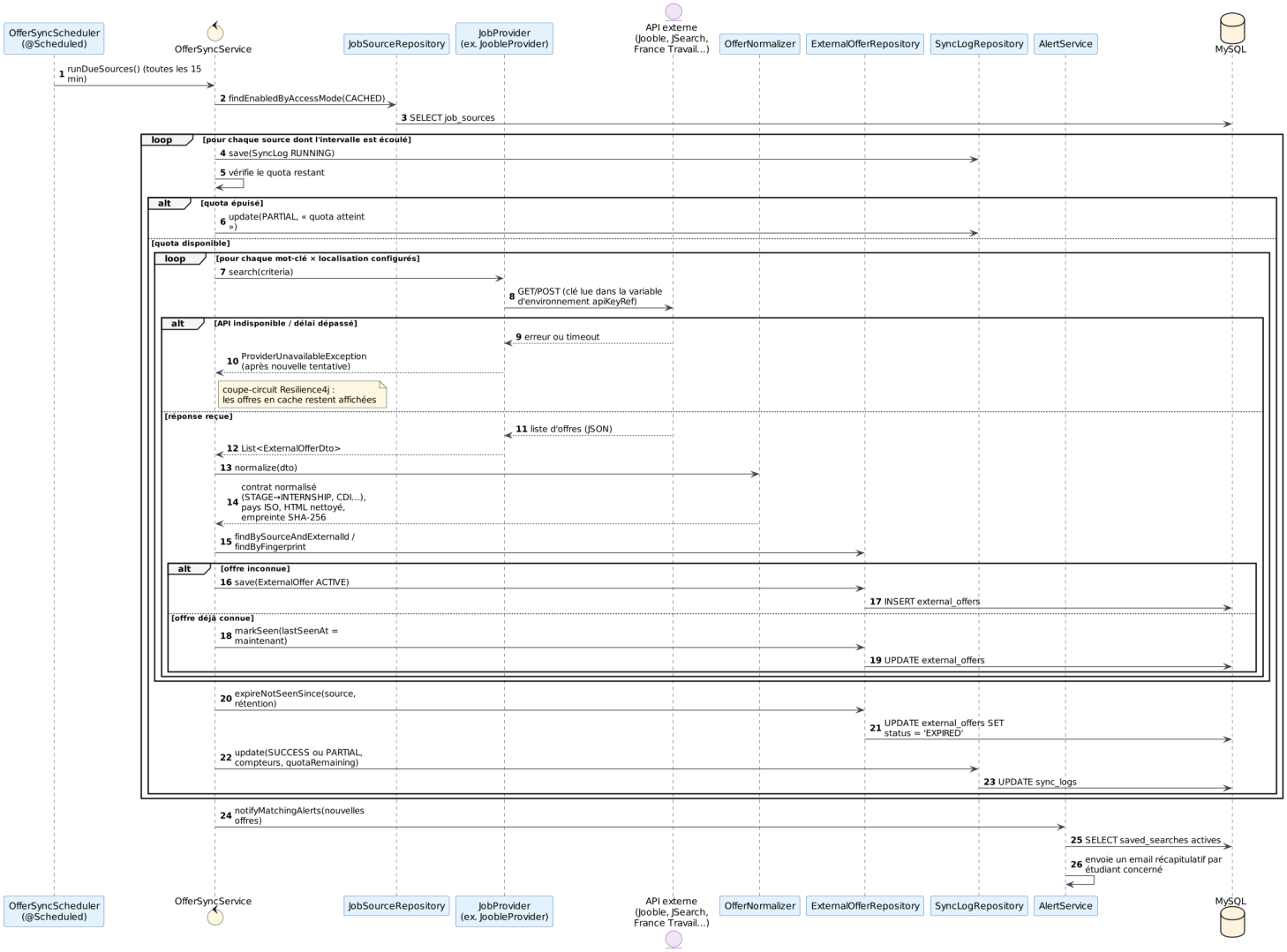
Séquence — Création d'une séance d'emploi du temps avec détection de conflits



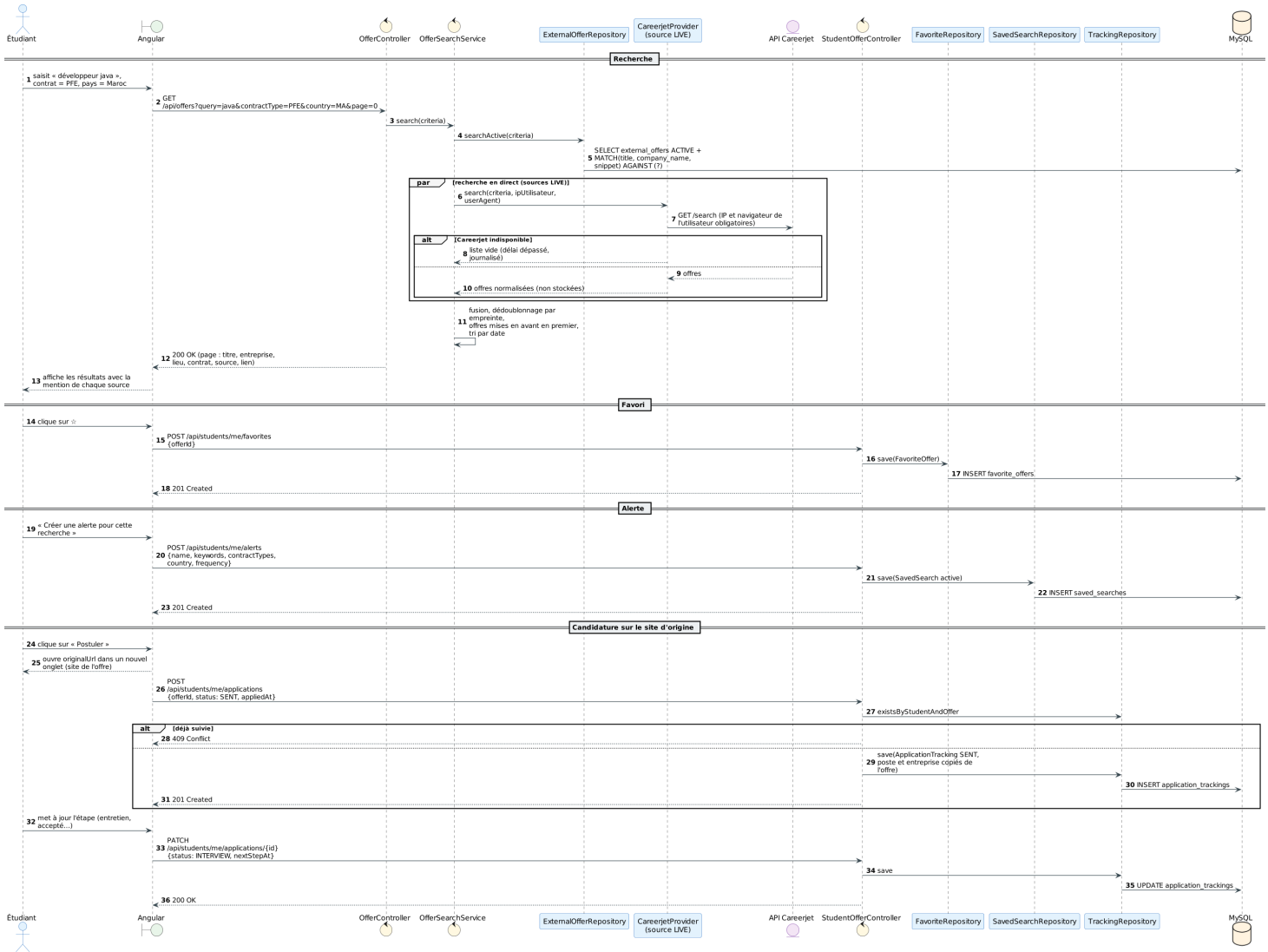

Séquence — Saisie, publication et consultation des notes
Enseignant
Étudiant
Angular
GradeController
GradeService
TeachingAssignment Repository
EnrollmentRepository
GradeRepository
AuditService
MySQL
Saisie des notes (enseignant)
1 ouvre une évaluation de son module
2 GET /api/examens/{id}/students
3 listStudents(examId, teacherId)
4 findEnrolledStudents(module, année)
5 SELECT enrollments + students + filiere_modules
6 liste des étudiants concernés
7 saisit les notes (0 à 20)
8 POST /api/examens/{id}/grades [{ studentId, score}]
9 saveGrades(examId, notes, teacherId)
10 existsByTeacherAndModule(teacherId, module)
alt [enseignant non affecté à ce module]
11 AccessDeniedException
12 403 Forbidden
[enseignant autorisé]
alt [note hors de l'intervalle 0 à 20]
13 ValidationException
14 400 Bad Request
[notes valides]
15 saveAll(notes au statut DRAFT)
16 INSERT ou UPDATE grades (unicité étudiant + évaluation)
17 log(GRADE_UPDATE)
18 200 OK
19 clique sur « Publier les notes »
20 PUT /api/examens/{id}/grades/ publish
21 publish(examId, teacherId)
22 updateStatus(examId, PUBLISHED)
23 UPDATE grades SET status = ' PUBLISHED'
24 log(GRADE_PUBLISH)
25 200 OK
Consultation (étudiant)
26 ouvre « Mes résultats »
27 GET /api/students/me/grades?year =2026-2027
28 findPublishedByStudent(userId, année)
29 findPublished(studentId, année)
30 SELECT grades PUBLISHED + vue v_student_module_averages
31 notes publiées et moyennes par module
32 affiche les résultats

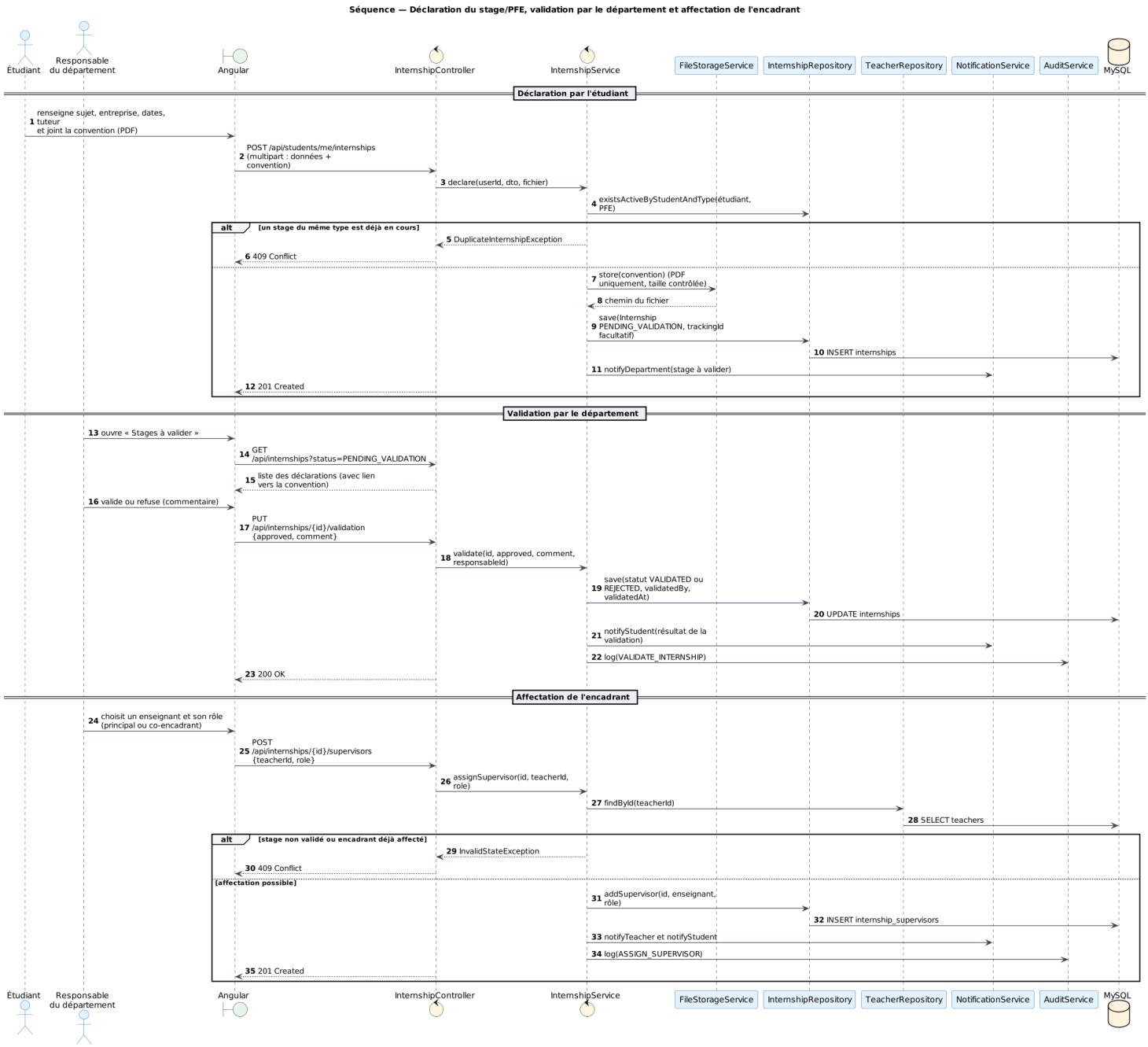

Séquence — Gestion des absences
Enseignant
Étudiant
Angular
AbsenceController
AbsenceService
PlanningRepository
EnrollmentRepository
AbsenceRepository
FileStorageService
AuditService
MySQL
Appel en début de séance
1 ouvre la séance du jour
2 GET /api/plannings/{id}/ attendance
3 getAttendanceSheet(planningId, teacherId)
4 findById(planningId)
5 SELECT plannings + teaching_ assignments
alt
[la séance n'est pas animée par cet enseignant]
6 AccessDeniedException
7 403 Forbidden
[séance de l'enseignant]
8 findStudentsByGroupe(groupeId)
9 SELECT enrollments + students
10 liste des étudiants du groupe
11 coche les absents et les retards
12 POST /api/plannings/{id}/ absences [{studentId, type}]
13 recordAbsences(planningId, liste, teacherId)
loop
[pour chaque étudiant signalé]
14 upsert(étudiant, séance, type)
15 INSERT ou UPDATE absences (unicité étudiant + séance)
16 log(RECORD_ABSENCES)
17 200 OK
Justification d'une absence
18 saisit le motif et joint le justificatif
19 PUT /api/absences/{id}/justify { motif, fichier}
20 justify(id, motif, fichier)
21 store(fichier)
22 chemin du fichier
23 save(is_justified = true, justification, justification_file)
24 UPDATE absences
25 200 OK
Consultation (étudiant)
26 ouvre « Mes absences »
27 GET /api/students/me/absences
28 findByStudent(userId)
29 findByStudent(studentId)
30 SELECT absences + séances + modules
31 liste (date, module, absence ou retard, justifiée ou non)
32 affiche ses absences

Séquence — Synchronisation planifiée des offres externes (sources CACHED)

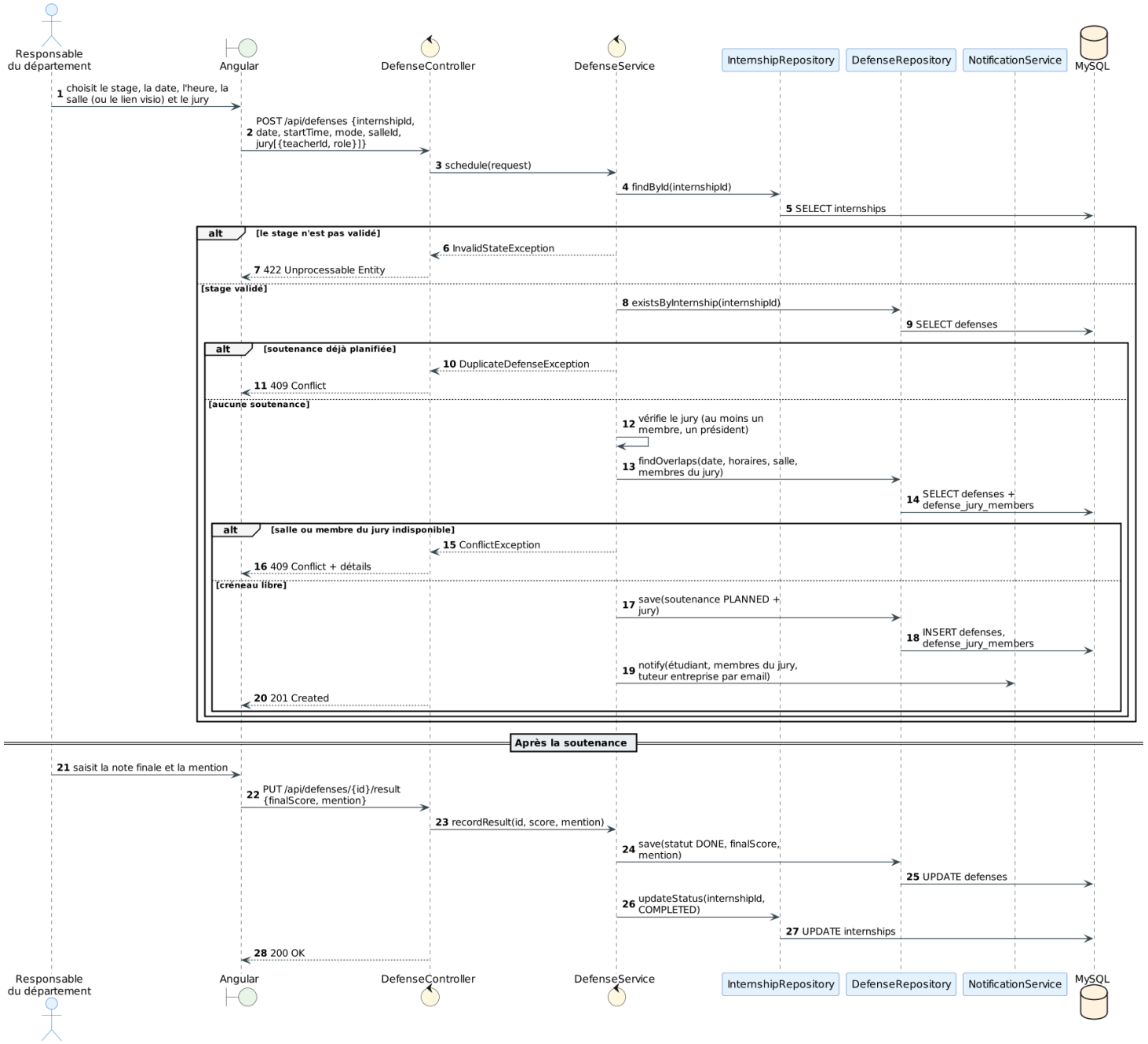


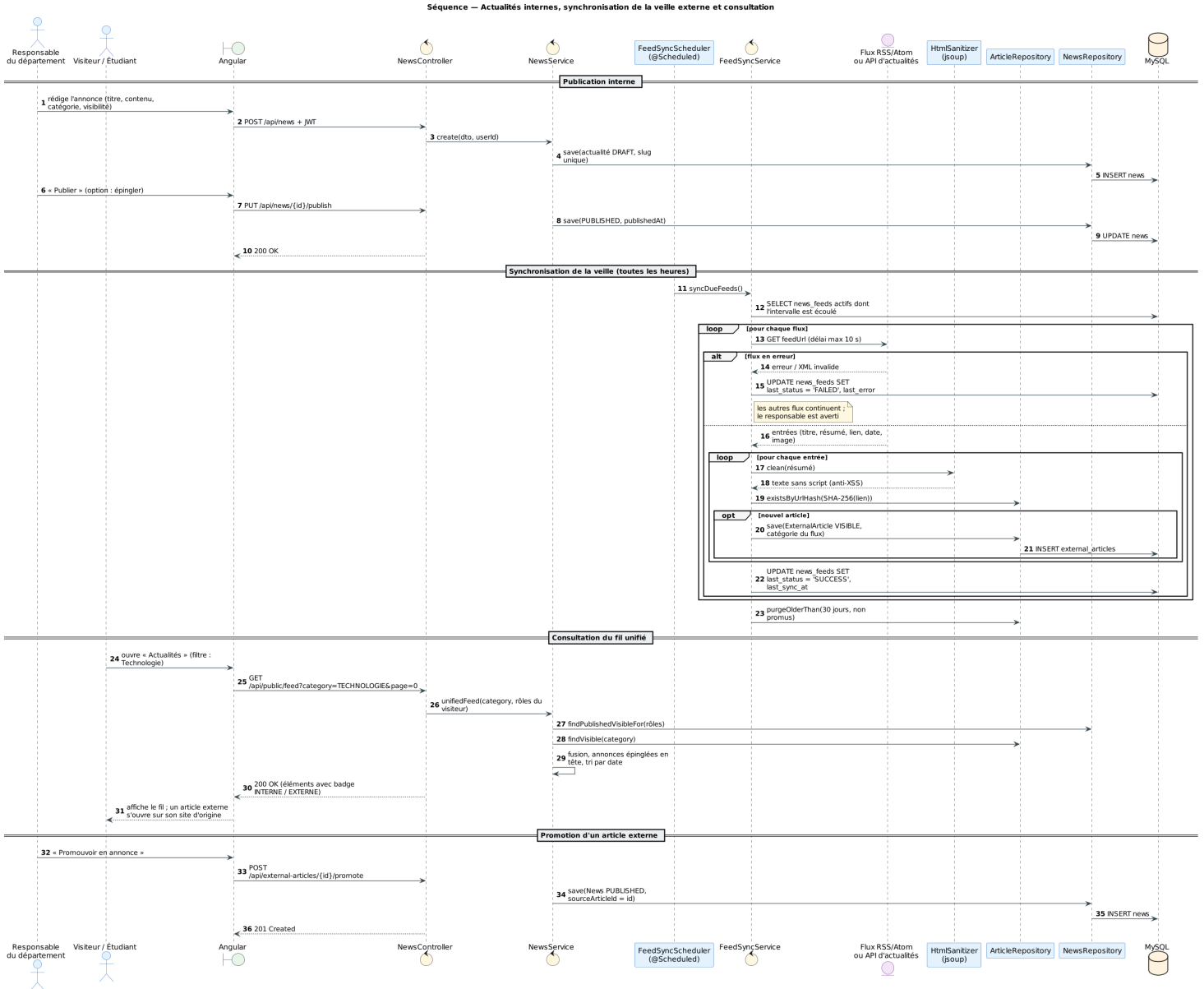
Séquence — Recherche d'offres, favoris, alerte et suivi de candidature (étudiant)




Séquence — Déclaration du stage/PFE, validation par le département et affectation de l'encadrant
Étudiant
Responsable du département
Angular
InternshipController
InternshipService
FileStorageService
InternshipRepository
TeacherRepository
NotificationService
AuditService
MySQL
Déclaration par l'étudiant
1 renseigne sujet, entreprise, dates, tuteur et joint la convention (PDF)
2 POST /api/students/me/internships (multipart : données + convention)
3 declare(userId, dto, fichier)
4 existsActiveByStudentAndType(étudiant, PFE)
alt [un stage du même type est déjà en cours]
5 DuplicateInternshipException
6 409 Conflict
7 store(convention) (PDF uniquement, taille contrôlée)
8 chemin du fichier
9 save(Internship PENDING_VALIDATION, trackingId facultatif)
10 INSERT internships
11 notifyDepartment(stage à valider)
12 201 Created
Validation par le département
13 ouvre « Stages à valider »
14 GET /api/internships?status=PENDING_VALIDATION
15 liste des déclarations (avec lien vers la convention)
16 valide ou refuse (commentaire)
17 PUT /api/internships/{id}/validation {approved, comment}
18 validate(id, approved, comment, responsableId)
19 save(statut VALIDATED ou REJECTED, validatedBy, validatedAt)
20 UPDATE internships
21 notifyStudent(résultat de la validation)
22 log(VALIDATE_INTERNSHIP)
23 200 OK
Affectation de l'encadrant
24 choisit un enseignant et son rôle (principal ou co-encadrant)
25 POST /api/internships/{id}/supervisors {teacherId, role}
26 assignSupervisor(id, teacherId, role)
27 findById(teacherId)
28 SELECT teachers
alt [stage non validé ou encadrant déjà affecté]
29 InvalidStateException
30 409 Conflict
[affectation possible]
31 addSupervisor(id, enseignant, rôle)
32 INSERT internship_supervisors
33 notifyTeacher et notifyStudent
34 log(ASSIGN_SUPERVISOR)
35 201 Created

Séquence — Planification d'une soutenance et enregistrement du résultat (v02)




Séquence — Actualités internes, synchronisation de la veille externe et consultation
Responsable du département
Visiteur / Étudiant
Angular
NewsController
NewsService
FeedSyncScheduler (@Scheduled)
FeedSyncService
Flux RSS/Atom ou API d'actualités
HtmlSanitizer (jsoup)
ArticleRepository
NewsRepository
MySQL
Publication interne
1 rédige l'annonce (titre, contenu, catégorie, visibilité)
2 POST /api/news + JWT
3 create(dto, userId)
4 save(actualité DRAFT, slug unique)
5 INSERT news
6 « Publier » (option : épingler)
7 PUT /api/news/{id}/publish
8 save(PUBLISHED, publishedAt)
9 UPDATE news
10 200 OK
Synchronisation de la veille (toutes les heures)
11 syncDueFeeds()
12 SELECT news_feeds actifs dont l'intervalle est écoulé
loop [pour chaque flux]
13 GET feedUrl (délai max 10 s)
alt [flux en erreur]
14 erreur / XML invalide
15 UPDATE news_feeds SET last_status = 'FAILED', last_error
les autres flux continuent ; le responsable est averti
16 entrées (titre, résumé, lien, date, image)
loop [pour chaque entrée]
17 clean(résumé)
18 texte sans script (anti-XSS)
19 existsByUrlHash(SHA-256(lien))
opt [nouvel article]
20 save(ExternalArticle VISIBLE, catégorie du flux)
21 INSERT external_articles
22 UPDATE news_feeds SET last_status = 'SUCCESS', last_sync_at
23 purgeOlderThan(30 jours, non promus)
Consultation du fil unifié
24 ouvre « Actualités » (filtre : Technologie)
25 GET /api/public/feed?category=TECHNOLOGIE&page=0
26 unifiedFeed(category, rôles du visiteur)
27 findPublishedVisibleFor(rôles)
28 findVisible(category)
29 fusion, annonces épinglées en tête, tri par date
30 200 OK (éléments avec badge INTERNE / EXTERNE)
31 affiche le fil ; un article externe s'ouvre sur son site d'origine
Promotion d'un article externe
32 « Promouvoir en annonce »
33 POST /api/external-articles/{id}/promote
34 save(News PUBLISHED, sourceArticleId = id)
35 INSERT news
36 201 Created